Kaarigar Setu

Security status: what works, what works only partly, and what is not built

CBS 3002 Information Security. Written 16 October 2026 from the state of the code and the test runs on that date. Everything marked 'works' was actually run; where something was not run, this document says so.

Status	Meaning	How many items
WORKS	We attacked it or tested it, and the defence held.	38 of 40 attacks blocked, plus tests, scans and design checks
WORKS ONLY PARTLY	A real protection exists, but it has a known limit that an attacker or an ordinary user can reach.	12 items (section 3)
NOT BUILT	Missing. Either an attack still succeeds, or the feature simply does not exist.	10 items (section 4)
NOT RUN HERE	Written and reviewed, but never executed in this environment, so no claim is made about it.	9 items (section 5)

Where to look: section 2 lists what works, section 3 what works only partly, section 4 what is not built, section 5 what was never run, section 6 what was found and fixed, section 7 the suggested next steps, **section 8 explains when each protection comes into play while the website runs**, and section 9 how to reproduce everything.

The numbers behind the headline

Check	Result on the final code
Attack-and-defence run (scripts/security_attack_demo.py)	40 attacks: 38 blocked, 0 vulnerable, 2 residual (known, accepted), 0 skipped
Java tests / console tests / Python tests	322 / 13 / 370, all passing
End-to-end checks against the running app / API checks	108 / 77, all passing
Static analysis (SpotBugs + security rules, Bandit, Semgrep)	0 open findings (85 raw SpotBugs findings: fixed or justified in writing)
Secret scan (gitleaks) / dependency scans (pip-audit, OSV)	no secrets in what ships; no known vulnerable dependencies
Software bill of materials	generated for the Java app (11 components) and the ML service (11 components)

Short version. The login system, access control, request forgery, injection, credential signing and verification, audit trail and trust-score protections all hold under attack. The two biggest honest weaknesses are that a worker is never asked to accept a rating an employer posts about them, and that fake employer accounts with their own phone numbers can still push a worker into the 'Good' band. Several things were designed and tested but never run for real (Docker, a real phone, real SMS, the language-model scorer).

1. How to read this document

The project in one paragraph. A tradesperson (electrician, plumber and so on) answers questions by voice; speech recognition and a scoring rule turn the answers into a skill score. Past employers confirm the worker's jobs, a trust score is built from those confirmations, and the worker gets a digitally signed credential with a QR code that anyone can check. A human moderator reviews anything the fraud model flags.

Words used below.

- **One-time code (OTP)**: the 6-digit code sent to a phone to prove you control it.
- **Session / cookie**: what keeps you signed in after you enter the code.
- **CSRF**: a trick where another website makes your browser send a request to this app while you are signed in.
- **IDOR**: reading someone else's data just by changing an id in a request.
- **Credential**: the signed document (plus QR code) that states a worker's trade, skill score and trust band.
- **Audit trail**: the permanent log of security-relevant events. It must be impossible to edit quietly.
- **Sybil attack**: creating many fake accounts to look like many independent people.
- **Vetting**: a moderator checking, outside the app, who an employer really is.
- **Residual risk**: a weakness we know about, have measured, and have chosen to document rather than fix now.

Where the evidence is. Attack IDs such as AUTH-01 or CRED-03 refer to scripts/security_attack_demo.py and the generated table in docs/PENTEST_RESULTS.md. Tests are named where they exist. The full reports are docs/PENTEST_REPORT.md, docs/SECURITY_ARCHITECTURE.md, docs/FRAUD_ATTACK_SIMULATION.md, docs/CREDENTIAL_TRUST.md and docs/SECURITY_SCAN_REPORT.md.

2. What works (attacked or tested, and the defence held)

2.1 Signing in and staying signed in

What an attacker tries	What happened	Proof
Guess the one-time code	Six wrong guesses all refused; after five failures the real code no longer works either (the code is burned).	AUTH-01
Use a code twice	The second use, from another client, is refused.	AUTH-02
Use a worker's code to sign in as an employer	Refused: a code is bound to the role and the phone number.	AUTH-03
Plant a known session id before login (session fixation)	The session id changes at login; the old one is useless.	AUTH-04, e2e (now a real check)
Reuse a stolen cookie after the user signs out	Refused: the session is destroyed on the server.	AUTH-05
Guess the moderator password	Throttled after 5 tries per 10 minutes. A real and an unknown username get the same answer and take the same time, so names cannot be discovered.	AUTH-06
Change the 'client address' header to dodge the code limit	The limit is also kept per phone number, so rotating the address does not help.	AUTH-07
Add fields like role or score to a request	Ignored; the account is created with default values.	AUTH-09
Flood sign-ups / find out which phone numbers are registered	Throttled: the 21st attempt from one address is refused.	AUTH-10

2.2 Who can see and do what (access control)

What an attacker tries	What happened	Proof
Read another worker's passport by changing the id (IDOR)	404, identical to a non-existent id, so it does not even reveal who exists.	ACL-01
Download or revoke someone else's credential	Refused for workers (404), employers (403) and anonymous callers (401); the victim's credential stays valid.	ACL-02
Every role calls every other role's endpoints	17 cross-role calls, every one refused with 401 or 403.	ACL-03
Open dashboards without signing in	Redirected or refused; never shows data.	ACL-04
Employer B confirms employer A's attestation with the right code	Refused (404); the real employer can still finish it.	ACL-05
Vouch for yourself using your own phone number	Refused, and a control request from an unrelated employer succeeds, so the refusal is for the right reason.	ACL-06
One employer pushes codes to many workers	Throttled at the 11th request.	ACL-07

2.3 Attacks through requests and pages

What an attacker tries	What happened	Proof
SQL injection in every public field, including a 6-second database sleep	30 hostile requests: no server error, no database error text, the sleep never ran. All database access uses prepared statements, enforced by a test that fails the build otherwise.	INJ-01, NoStringBuiltSqlTest, DaoSqlInjectionFuzzTest
Scripts and template expressions in a name field	Stored as plain text, escaped on output, expression not evaluated, strict content-security policy.	INJ-03
Path traversal and hidden files (WEB-INF, .env, audit log)	12 probes, none returned content.	INJ-04
Cross-site request forgery	No token, wrong token, another session's token and a foreign origin are all refused; the session survives.	CSRF-01
Malformed, huge or deeply nested request bodies	Clean 4xx errors, no stack traces.	DATA-01
Oversized or fake audio uploads	6 MB of zeros, a header claiming 2 GB and a Windows program renamed as audio are all refused.	DATA-02
Fingerprint the server or find phone numbers in responses	No version banner on the app's own responses; no phone number in any passport, credential list or public verification page.	DATA-03, DATA-04
Flood the public verification endpoint	Throttled at the 121st request in a minute.	DATA-05

2.4 The credential (the product itself)

What an attacker tries	What happened	Proof
Edit the score in a credential	Rejected, both by the server and by the independent offline verifier.	CRED-02
Sign a forged credential with their own key	Rejected: the signature must come from a trusted key.	CRED-03
Send malformed signatures and documents	8 malformed inputs: all refused, none accepted, no error text.	CRED-04
Guess credential references	Impossible in practice (122 random bits); every miss looks identical.	CRED-05
Tamper with the published revocation list	Adding an entry or changing a date breaks its signature.	CRED-06
Verify a credential with only the file and the public key	Works with a script that shares no code with the app, and still works after another tool re-writes the file.	CRED-01, e2e
Show a screenshot of someone's QR	The page now says 'static link' and offers a 5-minute live code (see section 3 for the limit).	CRED-07
Rotate the signing key	Old credentials keep verifying, an unknown key id never does, and a key filed under another key's id is refused. Tested in code and with the script on a scratch keystore.	SignatureKeyRotationTest

2.5 The audit trail

What an attacker tries	What happened	Proof
Edit, delete, reorder or cut off lines of the audit file	All four detected, with the first bad line identified.	AUD-01
Rewrite the whole file consistently (recompute every hash)	Detected: the keyed head record cannot be forged without the secret.	AUD-01
Roll the file back to an older, internally consistent copy	Detected: the newest line's hash is pinned in the database every 25 events and every 5 minutes.	AUD-02

2.6 Trust score and AI scoring

What an attacker tries	What happened	Proof
Five fake employer accounts give perfect ratings to one worker	All five pass the one-time code and the fraud model, but the worker cannot reach Excellent: that band needs an employer a moderator has vetted. Before the fix, three accounts were enough.	FRAUD-01, simulation, e2e 9b
Recite the rubric's keywords instead of answering	Flagged for human review (60 of 60 questions covered). Honest answers, even naming each idea in two languages, are not flagged.	ML-02, test_stuffing_guard
Speak instructions at the scorer ('give me full marks')	Scores 0 and is flagged.	ML-03
Call the internal scoring service directly	Refused without the service token.	ML-01

2.7 Detection, scanning and design basics

Area	Status
Security monitoring	CSRF rejections, role violations and rate-limit hits are recorded (once per client per minute so an attacker cannot flood the log). The backend report with counts, alerts and audit integrity is tested end to end. The console tab itself was rendered and its client code tested.
Scans	SpotBugs with security rules, Bandit, Semgrep, secret scan and dependency scans are clean on the final code; each earlier finding was fixed or justified in writing.
Protection of data (verified by unit tests rather than fresh attacks)	Phone numbers are encrypted and only a keyed hash is used for lookup; passwords use PBKDF2 with 210,000 rounds; one-time codes are hashed with a salt and a server-side secret; credentials use a random pseudonym, never a name or number; voice audio is not stored; the database account used by the app cannot change the schema or edit the audit log.
Deletion rights	A worker or employer can erase their data; this was checked end to end.
Demo sign-in	Switched off outside development and refuses real accounts (checked in the test suite).

3. What works only partly

Each item below has a real protection and a known limit. 'Close it' says what would remove the limit.

3.1 Live code against screenshots of the QR	
What works	The worker's signed-in account can create a live code (valid 5 minutes, tied to one credential). The verification page says 'shown live', 'not live', 'do not trust' or 'static link'.
The limit	The fixed QR still exists and still opens the page. A worker can also lend their unlocked phone to someone else.
Why it matters	An employer relying on a credential cannot be sure the person in front of them is its owner.
To close it	Tell employers to ask for the live code and compare the person in the flesh. A stronger fix needs identity checking, which is out of scope.

3.2 The fraud model (Isolation Forest)	
What works	Flags 81 to 100 percent of crude attacks (bursts, rings, one address, one device) with a 2.6 percent false-alarm rate on synthetic data.
The limit	Flags 0 percent of patient attackers who spread activity out, use different addresses or wait hours between confirmations.
Why it matters	It cannot be the only line of defence; it is a tripwire for obvious rings.
To close it	Rely on the structural rules (distinct employers, vetting) and retrain on real pilot data. Address and device signals are also weak on mobile networks where many phones share an address.

3.3 Fake employers can still reach the 'Good' band	
What works	The top band (Excellent) now needs an employer a moderator has vetted. A ring of five unvetted accounts is held at 79.99.
The limit	Three fake employers, each with its own SIM card, still give a worker a Good score (up to 79.99).
Why it matters	A determined worker could present a 'Good' trust band that was bought.
To close it	Raise the cost per fake employer (vetting for the Good band too, or a probation period for new employers), and review new employers' first confirmations by hand.

3.4 Vetting depends on a person	
What works	Vetting is recorded with who and when, audited, and visible in the console. Without it the top band is out of reach.
The limit	A fooled or corrupt moderator can vet a fake employer. There is one shared moderator account, so the trail cannot say which person did it.
Why it matters	The trust anchor is only as honest as the moderators.
To close it	Named moderator accounts, two-person approval for vetting, and a documented checklist for what vetting means.

3.5 Speech recognition accuracy, especially accents	
What works	Hindi speech is transcribed and scored. The memory-light engine matches the heavier one closely. The accuracy was measured honestly on real voices.
The limit	On 60 real speakers the word error rate is about 32 percent (character error about 17 percent), and 3 of 60 clips came back empty. No accent training was done: that needs a graphics card and consented recordings.
Why it matters	A worker with a strong accent may be scored too low, which is unfair and also an integrity problem for the skill score.
To close it	Try a Hindi-tuned model (the download was lost when temporary files were cleaned), then collect consented pilot recordings and fine-tune. The plan is in docs/ACCENT_TRAINING.md.

3.6 Keyword scoring is gameable in principle	
What works	The default scorer awards marks for expected words. Recitation and spoken instructions are caught.
The limit	Any keyword scorer can be gamed in ways nobody has thought of yet. The model-based scorer would judge meaning better but needs an API key and was never run.
Why it matters	The skill score is only as good as the scorer.
To close it	Run the model-based scorer with the existing robustness tests, and have a trade expert validate the rubrics (they are placeholders marked TEST ONLY).

3.7 Moderator console tabs	
What works	The Security and Employers tabs were built, rendered with demo data, and their server calls are tested. The backend behaves correctly end to end.
The limit	Nobody has clicked through the tabs against the live server in the real window.
Why it matters	A layout or interaction bug could still be hiding.
To close it	A short manual run-through of the console: sign in, open each tab, vet an employer, verify the audit log.

3.8 Rate limits keyed by client address	
What works	Limits per phone number, per username and per code always hold.
The limit	On the development setup the app trusts a client-address header so tests can simulate many clients; a directly reachable app would let a caller choose that address.
Why it matters	Address-based limits can be dodged if the app is exposed without the proxy that overwrites the header.
To close it	Set TRUST_FORWARDED_FOR to false unless the proxy is in front. This is a one-line production setting and is written in the runbook.

3.9 Development server shows its default error page	
What works	The production configuration (docker/tomcat/server.xml) hides the version and the error details.
The limit	The development server started by the build tool shows Tomcat's default page, including its version, for a few malformed addresses it rejects itself.
Why it matters	Minor information disclosure in development only.
To close it	Use the hardened server configuration for development too, or run the Docker image.

3.10 Key rotation was tested on a scratch keystore	
What works	The code path and the script both work; old credentials stay valid.
The limit	The live development key has never been rotated, so the full procedure was not performed on real data.
Why it matters	A rotation mistake in production would be discovered in production.
To close it	Do one rehearsal rotation on a copy of the database before launch.

3.11 Security monitoring thresholds are untuned	
What works	Counts, alerts and the audit-integrity line work.
The limit	The alert thresholds (for example 10 failed sign-ins in an hour) are reasonable guesses with no real traffic behind them.
Why it matters	Too sensitive means ignored alerts; too lax means missed attacks.
To close it	Watch the pilot's first weeks and adjust the thresholds in SecurityMonitor.RULES.

3.12 Audit writes re-read the whole file	
What works	Every append is locked, chained and fsynced; it is correct.
The limit	Each write re-reads the entire file, so cost grows with the log.
Why it matters	Fine for a pilot, slow after millions of lines.
To close it	Keep the last line and counter in memory and verify on startup.

4. What does not work or is not built

These are listed roughly from most to least important for the project.

#	Missing item	What goes wrong without it	What it would take
1	**Worker acceptance and dispute of a confirmation**	The one-time code goes to the employer's own phone, not the worker's, and the worker is never asked to accept. An employer can post a rating, including a bad one, about any registered worker, and the worker cannot object. This also means the old documentation was wrong when it said the worker's presence was proven (corrected).	Medium: show pending confirmations to the worker, let them accept or dispute, hold disputed ones for a moderator, exclude them from the score until resolved.
2	**A fix for targeted sign-in lockout** (AUTH-08)	Anyone who knows a phone number can use up its three code requests and block that person from signing in for about 10 minutes.	Small to medium: a per-device quota or a CAPTCHA, or a support-assisted recovery path.
3	**Named moderator accounts and two-person approval**	All moderator actions share one account; decisions and vettings cannot be attributed to a person.	Medium: accounts per moderator, roles, and a second approver for vetting.
4	**Real SMS delivery**	In development the one-time codes are written to the server log, so anyone with log access can read every code. Real users cannot receive codes at all.	Small: choose a gateway, set credentials, test delivery and failure handling (docs/SMS_SETUP.md).
5	**Response to a stolen signing key**	There is no bulk revoke of everything a compromised key signed, and credentials carry no independent trusted timestamp.	Medium: a revoke-by-key command, publish compromised key ids in the revocation list, add a timestamp service.
6	**Replayed or synthetic voice detection**	Someone could play a recording of an expert, or the same recording for several questions; the system treats it as the worker's own answer.	Large: audio fingerprinting, liveness prompts, anti-spoofing models.
7	**Encryption of the disk, database and backups**	Only phone numbers are encrypted at field level. Other data, the audit file and backups rely on the host's protection.	Medium: disk and database encryption, encrypted and tested backups.
8	**Accent-aware speech model and consent to train on voices**	The accuracy gap (about a third of words wrong on real voices) remains. Training on workers' recordings would need separate, withdrawable consent that does not exist yet.	Large: consent flow, collection, transcription, fine-tuning on a rented graphics card.
9	**Rate limits shared across several servers**	Limits are kept in each server's memory, so running two copies doubles every limit and a restart clears them.	Medium: move the limiter to the database or a shared cache before scaling out.
10	**Notification to a worker when an employer confirms**	Workers only learn of a confirmation by opening their dashboard.	Small once the acceptance flow in item 1 exists.

5. Written, but never run here

No claim is made about these. They are listed so nobody mistakes 'reviewed' for 'verified'.

Item	Why it was not run	What to do
Docker and the Caddy HTTPS proxy (the production path)	Docker is not installed on the development machine.	Run docker compose up on a machine that has Docker and repeat the end-to-end suite against it.
GitHub Actions workflow (static analysis, scans, attack run, ZAP baseline)	The repository has never been pushed to GitHub.	Push and watch the first run; fix whatever the runner finds.
OWASP ZAP dynamic scan and Trivy container scan	They run inside the CI workflow only.	Read their reports after the first CI run.
The language-model scorer	No API key is set.	Add the key and run scripts/llm_check.py and the robustness tests.
Real SMS delivery	No gateway account.	See item 4 above.
A real phone scanning the QR code	Not tried.	Scan from a phone on the same network.
Recording audio from a phone over HTTPS	The self-signed HTTPS helper was tested with scripted requests only; the in-app browser refuses self-signed certificates.	Open the helper's address on a phone and record an answer.
Real tradespeople's speech and accents	All speech tests used one synthetic voice or a public read-speech test set.	Pilot recordings with consent.
Interactive console against the live server	Not clicked through.	Manual run-through (section 3.7).

6. Weaknesses found by attacking the project, and fixed

This list is the evidence that the attack work was real: every row was a genuine weakness or defect before it was fixed.

Found	What was wrong	Fix
Fake employers (simulation, FRAUD-01)	Three brand-new fake employer accounts gave a worker an Excellent score; 'verified employer' was granted automatically.	Verified now means vetted by a moderator; the top band needs one; moderator list and vet action; worker told why.
Offline verification (CRED-01)	Only the original software could verify a credential, because the signature covered text like 81.30 and every other tool writes 81.3.	Canonical (RFC 8785) signing, key list, signed revocation list, independent verifier, key rotation.
Audit rollback (AUD-02)	An older, matching log and head record passed every check.	Database anchors every 25 events and 5 minutes; offline checker with database anchors.
Registration flood (AUTH-10)	Unlimited sign-ups, and duplicates answered 409, revealing which numbers exist.	20 registrations per 10 minutes per address.
QR replay (CRED-07)	A screenshot of the QR worked for anyone forever.	Live code and a verification page that says whether the code is live.
Rubric recitation (ML-02)	Reciting the rubric's words scored like a real answer; the first guard was too weak and the same test caught that.	Topic words and shared words judged separately; flagged answers go to a human.
Public verification flood (DATA-05)	No per-address limit.	120 requests per minute per address.
Static analysis	A password tool read input in the machine's default encoding, an unbounded regex input, a risky constructor, broad exception catches, a dead field.	Each fixed; the build now fails on any new finding.

Found	What was wrong	Fix
Time zones	Between midnight and 05:30 India time an employer could not record work done 'today'.	One day of slack on the work date.
Missing script	The audio converter file was lost when the project was rebuilt, so every recording ended in an error.	File restored, clearer messages, and a test fails the build if a page references a missing file.
Documentation	It said the worker receives the attestation code and so proves presence; the code sends it to the employer.	Corrected; recorded as open item 1.
The tests themselves	Early attack tests passed for the wrong reason (missing token, bad body); one end-to-end check could never fail.	Each test now includes a control that must succeed; the always-true check was replaced.

7. Recommended order of work

Order	Do this	Why first	Effort
1	Worker acceptance and dispute of confirmations (section 4, item 1)	It is the biggest open integrity and privacy gap, and the one most likely to be raised in a review.	Medium
2	Manual run-through of the console and a phone scan / recording over HTTPS	Converts several 'never run' items into verified ones in an hour.	Small
3	Push to GitHub and read the first CI run (including ZAP and Trivy)	Runs the checks that cannot run on this machine.	Small
4	Real SMS delivery, then named moderator accounts with two-person vetting	Removes the log-readable codes and the shared account.	Medium
5	Targeted lockout fix (per-device quota or CAPTCHA)	Closes the one residual attack on sign-in.	Small to medium
6	Hindi-tuned speech model, then consented pilot recordings	Improves fairness and scoring integrity for different accents.	Large
7	Model-based scorer with the existing robustness tests	Reduces how easily keyword scoring can be gamed.	Small to medium (needs a key)

8. When each protection comes into play while the website runs

The protections do not all act at once. Some run when the server starts, some on every single request, some only at one step of a user journey, some in the background, and some only when somebody checks. This section follows the website through its life in that order.

8.1 When the server starts (before any user arrives)

Moment	What acts	What it prevents
The app starts	Settings are validated: missing secrets, weak keys or unsafe production settings stop the start-up. The signing keystore and any retired public keys are loaded; a retired key filed under the wrong name is refused.	Running with missing or default secrets; a swapped-in key.
The app starts	Cookie rules are set once: HttpOnly, SameSite=Strict, Secure outside development, 30-minute idle timeout.	Scripts reading the session cookie; cookies sent from other sites.
The app starts	A background timer is started: it records the audit file's newest line in the database every 5 minutes.	A rolled-back audit file.
The ML service starts	Its shared-secret token is required; the fraud model file is checked against its recorded digest before it is loaded; the speech model is loaded.	Anyone else calling the scoring service; a poisoned model file.
Deployment (production)	The hardened web-server configuration hides the version, closes the shutdown port and trims error pages; the database account used by the app cannot change tables or edit the audit log.	Information leaks; a stolen app login being able to rewrite history.

8.2 On every single request (page or API call), in this order

Step	What acts	What it does
1	**Security headers** (all responses)	Adds the content-security policy (no inline scripts), clickjacking and content-type protections, and HSTS outside development.
2	**CSRF check** (only requests that change something)	A POST, PUT, PATCH or DELETE must carry the session's secret token and come from this site's own origin. Otherwise 403, and a CSRF_REJECTED event is written to the audit trail.
3	**Access control** (every page and API route)	Looks up the route in a deny-by-default table: is anybody signed in, has the 8-hour limit passed, does their role allow this route? Otherwise 401 or 403. A signed-in user reaching for another role's route is recorded as ACCESS_DENIED.
4	**Throttles** (public routes: sign-up, verification, QR, keys)	Counts requests per client address in a sliding window. Over the limit: 429 with a retry time, and a RATE_LIMITED event (at most one per client per minute so attackers cannot flood the log).
5	**Input checks and error handling** (the servlet itself)	Bodies must be small JSON; every database call uses prepared statements; every page escapes output. Any failure becomes a short coded error, never a stack trace.

8.3 At each step of the user journey

Journey step	Protections that act at that moment	Written to the audit trail
Registering (worker or employer)	Sign-up throttle (20 per 10 minutes per address); the consent box must be ticked or nothing is stored; the phone number is encrypted and a keyed hash is kept for lookup; extra fields in the request are ignored.	WORKER_REGISTERED, EMPLOYER_REGISTERED
Asking for a sign-in code	Limits of 3 per phone number and 10 per address per 10 minutes; the reply is identical whether or not the number is registered; the code is stored only as a salted, peppered hash and expires in 5 minutes.	LOGIN_OTP_REQUESTED (or ..._UNKNOWN)
Entering the code	At most 5 attempts, then the code is burned; single use; bound to the role and the number; on success a brand-new session id and a new CSRF token are issued.	LOGIN_SUCCESS / LOGIN_FAILED
Moderator sign-in	Password (PBKDF2, 210,000 rounds) then a second code; an unknown username costs the same time as a wrong password; 5 attempts per username per 10 minutes.	MODERATOR_OTP_ISSUED, LOGIN_*
Starting the voice test	Worker role required; an attempt token is signed so a forged one is refused; only question text and marks are sent to the browser, never the grading keywords.	ASSESSMENT_STARTED
Uploading each spoken answer	Before anything else: size cap, WAV header and real duration are checked and a fake or oversized file is refused. The audio stays in memory and is thrown away after transcription. The scoring service is called with its shared token.	(audio is never stored)
Turning speech into a score	The transcript is cleaned (invisible characters stripped, length capped); injection phrases and recited rubric words raise a human-review flag; the marks the scorer returns are clamped and the totals recomputed by our code; if the scoring service is down nothing is auto-approved.	ASSESSMENT_COMPLETED
Issuing the credential	Needs a complete test with no answer under review and enough trust; the document is written in canonical form, signed with the RSA key whose id is recorded, and every earlier credential of that worker is revoked.	CREDENTIAL_ISSUED
Showing the credential	Download and live code are owner-only (anyone else gets 404). The live code is a 5-minute token tied to one credential.	CREDENTIAL_LIVE_CODE
Someone checks the credential (QR or link)	Throttled per address; the server re-checks signature, the key named in the credential, revocation and expiry every time; the page says whether the code shown is live.	(public, not logged)
Employer confirms a job	Employer role required; work date must be plausible; refused if it is your own number; one live confirmation per pair; 10 per employer and 30 per address per 10 minutes; the code goes to the employer's own phone and the same code rules apply.	ATTESTATION_OTP_REQUESTED / _FAILED
After the employer's code is entered	The fraud model scores the confirmation. Crude patterns go to a moderator; otherwise it is approved, the trust scores are recalculated (including the rule that the top band needs a vetted employer) and the worker's credential is re-issued. If the fraud model is down it goes to a human.	TRUST_RECOMPUTED, CREDENTIAL_ISSUED
Moderator decisions	Moderator role and CSRF token required; a flag can be decided only once; vetting an employer is recorded with who and when and triggers a score refresh.	MODERATOR_DECISION, EMPLOYER_VETTED
Deleting an account	The person must type the confirmation phrase; personal data and credentials are removed, and the session ends.	WORKER_ERASED, EMPLOYER_ERASED

8.4 In the background, while nobody is doing anything

When	What acts
Every 25 audit events and every 5 minutes	The newest audit line's hash is recorded in the database (the anchor).
Every 30 minutes	Old rate-limit counters are cleaned out of memory.
After each approval, ruling, vetting or erasure	All trust scores are recalculated and affected credentials re-issued.

8.5 Only when somebody looks

Who looks, and when	What acts
A moderator opens the Security tab, or presses 'Verify audit log'	The audit file is checked against its chain, its keyed head record and the database anchors; the database's own chain is checked; counts of attack-like events in the last hour and day are compared with thresholds and turned into alerts.
An employer or bank checks a credential offline	scripts/verify_credential.py rebuilds the signed bytes, checks the signature and key, the dates and (optionally) the signed revocation list, without contacting the app.
An auditor suspects tampering	AuditVerifyCli checks a copy of the audit file offline, with the database anchors if given.
An operator rotates the signing key	The old public key is kept so old credentials stay valid; new ones use the new key.

8.6 Before the website is even deployed (build time and CI)

When	What acts
Every build	A test fails the build if any database query is built by joining strings; another fails it if a page refers to a script or style file that does not exist; the full test suite runs.
Every push (CI, once the repository is on GitHub)	SpotBugs with security rules, Bandit, Semgrep, a secret scan, dependency scans, a software bill of materials, the attack-and-defence run and a passive ZAP scan. None of this has run in CI yet, only locally.
On demand	scripts/security_attack_demo.py attacks a running development stack with 40 real attacks.

8.7 One example, end to end

A worker's whole day. At start-up the keystore and secrets were checked. The worker registers: the sign-up throttle and consent rule act. They ask for a code and enter it: limits, hashing, a fresh session. Every request they make passes security headers, the CSRF check and the role table. They record ten answers: each upload is size- and format-checked, transcribed from memory, cleaned and scored with clamped marks. They issue a credential: it is signed and logged. An employer confirms a job: a second code, a fraud-model check and a trust recalculation. A stranger scans the QR: the server re-verifies the signature, the key, revocation and expiry. In the background the audit trail is anchored. If at any point something looked like an attack (a forged token, a role violation, a flood), the moderator's Security tab would show it.

9. How to reproduce the claims in this document

To see	Run (from the project folder, with the development stack running)
All 40 attacks	BASE_URL=http://localhost:8090 JAVA_LOG=data/logs/dev.log python3 scripts/security_attack_demo.py
Verify a credential offline	python3 scripts/verify_credential.py credential.json --server http://localhost:8090
Fake-employer simulation	from kaarigarsetu-java: java -cp "target/classes:target/test-classes:$(cat target/demo.classpath)" com.kaarigarsetu.trust.TrustAttackSimulation
Static analysis and bill of materials	cd kaarigarsetu-java && mvn -Psast -DskipTests compile spotbugs:check cyclonedx:makeAggregateBom
Audit file check (offline)	AUDIT_HMAC_KEY=... java -cp ... com.kaarigarsetu.admin.AuditVerifyCli data/audit.log --db
Watch one-time codes	./scripts/otp.sh -f
Everything end to end	STT_PROVIDER=stub ./scripts/dev.sh, then python3 scripts/e2e.py (108 checks)

Moderator sign-in allows 5 attempts per 10 minutes, and the test scripts share that allowance; restart the app to clear it. The attack script creates test accounts and sends hostile input, so it must only be pointed at a development stack.